



Scope Document

AI-Based City Pathfinding and Visualization System

Name	Ali Haider Aimen Shahzad
Reg No	FA23-BSE-014 FA23-BSE-111
Subject	Artificial Intelligence
Instructor	Mr. Waqas Ali

1. Project Overview

The **AI-Based City Pathfinding and Visualization System** is an educational and practical web-based application designed to demonstrate the working of Artificial Intelligence search algorithms—specifically **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**—in a **real-world context of cities and roads**.

The system enables users to **create virtual city maps**, where each city represents a **node**, and roads represent **edges** connecting these nodes with **distances as weights**. By applying search algorithms, users can visualize how AI explores possible routes between two cities, identifies reachable paths, and determines traversal order.

Developed using **Flask (Python)** as the backend and **Vis.js (JavaScript)** as the frontend visualization library, the system bridges theory with practice by enabling users to interactively build graphs, visualize traversal animations, and analyze algorithmic behavior step-by-step.

This project emphasizes **Artificial Intelligence in problem-solving**, particularly the **state-space search paradigm**, **graph-based reasoning**, and **heuristic exploration concepts**, laying the foundation for understanding advanced algorithms such as **Dijkstra's**, **A***, and **Greedy Best-First Search**.

2. Project Motivation

In Artificial Intelligence, search algorithms are fundamental techniques used for problem-solving in a variety of domains such as robotics, route planning, gaming, and decision-making. However, many students find it difficult to visualize how algorithms explore and make decisions across complex networks.

This project was motivated by the need to:

- Transform **abstract AI search concepts** into a **tangible, real-world scenario**.
- Build an **interactive simulation tool** for AI students to understand BFS and DFS through visualization.
- Demonstrate **pathfinding as a real-world application of AI**—specifically in **transportation systems** and **smart city navigation**.

By modeling cities and their connections as a search problem, this system provides a clear bridge between **graph theory**, **AI search mechanisms**, and **real-world navigation**.

3. Objectives

Primary Objectives

1. To design and implement a **visual platform** that demonstrates how AI algorithms traverse city maps.
2. To apply **BFS** and **DFS** for **pathfinding** between cities using real-world analogies.
3. To provide **step-by-step visualization** of AI decision-making during traversal.
4. To allow **user interaction** for creating custom city networks and testing algorithms.

Secondary Objectives

1. To store and retrieve city maps for repeated experimentation.
2. To enable comparison between algorithms in terms of **visited nodes**, **path length**, and **exploration order**.
3. To provide a base system that can be extended to **heuristic algorithms** like **A*** and **Greedy Search**.
4. To integrate core **AI search principles** (state-space representation, transitions, goal-testing) into an interactive simulation.

4. Problem Statement

In AI education, understanding how search algorithms like BFS and DFS work is often limited to static examples on paper or console-based outputs. Such representations do not effectively capture:

- The **order of exploration** of nodes,
- The **structure of search trees**, or
- The **realistic representation** of movement between connected locations.

This project addresses this limitation by developing a **web-based AI visualization system** that represents cities and roads as graph structures and demonstrates algorithmic exploration dynamically.

It enables learners to **observe AI pathfinding behavior** in real-time, helping them understand how algorithms make decisions in a spatial, real-world context.

5. Project Scope

The project is primarily focused on **AI search algorithm visualization** applied to **pathfinding in cities**. It allows users to:

- Create and visualize a map of cities (nodes) and their connections (edges).
- Select and execute AI algorithms such as BFS and DFS.
- Observe step-by-step traversal with animations and path highlights.
- Analyze search patterns, path cost, and order of exploration.
- Save and load graph configurations for future use.

While the current version supports BFS and DFS, the architecture is designed for future expansion into **informed search algorithms** such as:

- **Uniform Cost Search**
- **Greedy Best-First Search**
- **A* Search**

The system thus lies within the **Artificial Intelligence – Search and Problem Solving domain**, emphasizing both the **visual and logical understanding** of AI-based decision-making.

6. System Description

6.1 Functional Overview

The system provides a dual-layer structure:

1. **Frontend (Visualization Layer)** — implemented using **HTML, CSS, JavaScript, and Vis.js**.
2. **Backend (Algorithmic Layer)** — implemented using **Python (Flask Framework)**.

The frontend allows users to create nodes and edges (cities and roads) and visualize the AI algorithms' operation. The backend processes algorithmic computations and responds with data for visualization.

6.2 Functional Features

Frontend (Vis.js Interface)

- Interactive graph creation: Add and connect cities visually.
- Algorithm selection and execution (BFS/DFS).
- Real-time visualization of node exploration.
- Distinct color coding:
 - **Start Node:** Green
 - **End Node:** Red
 - **Visited Nodes:** Yellow/Blue
 - **Final Path:** Highlighted in bold line or color
- Animated transitions showing AI exploration.
- Graph control options (clear, save, load, toggle physics).

Backend (Flask Server)

- Graph data management using adjacency lists.
- API endpoints for:
 - Adding nodes and connections
 - Computing BFS or DFS paths
 - Returning traversal sequence and parent-child data
 - Saving/loading graphs as JSON files
- Computation of path cost (sum of edge distances).
- Error handling for invalid inputs or missing data.

7. Artificial Intelligence Integration

The project demonstrates **AI search concepts** in a real-world domain (city navigation):

AI Concept	Implementation
State Space Representation	Each city represents a state; moving from one city to another represents a transition.
Search Algorithms	Implemented BFS and DFS to explore reachable paths.
Goal Test	Reaching the destination city satisfies the goal condition.
Search Tree Visualization	The exploration order is represented visually as an animated search tree.
Path Cost Calculation	Each road has a distance that contributes to the total cost.
Agent Simulation	The “AI agent” explores nodes step by step, emulating autonomous pathfinding behavior.

This approach transforms abstract AI theory into an **interactive agent-based model**, helping students visually grasp **uninformed search strategies** and their implications in practical navigation problems.

8. System Architecture

8.1 Architecture Overview

The system follows a **Client-Server Architecture** using RESTful communication.

Client (Frontend):

- Provides an interactive interface for visualization and control.
- Communicates with Flask backend via HTTP requests.
- Displays results in real-time using Vis.js.

Server (Backend):

- Performs all algorithmic computations.
- Maintains and manipulates the graph data structure.
- Handles graph storage, loading, and API routing.

8.2 Data Flow

1. User creates or loads a city graph from the frontend.
2. The frontend sends requests (via `fetch`) to Flask endpoints (e.g., `/api/add_city`, `/api/find_path`).
3. Flask computes the result (e.g., BFS path) and sends back JSON data.
4. The frontend updates the visualization, highlighting visited nodes and final paths.
5. User can save graphs, clear the view, or switch algorithms.

9. Technologies Used

Category	Technology
Programming Language	Python (Flask), JavaScript (ES6)
Frontend Libraries	Vis.js, HTML5, CSS3
Backend Framework	Flask (RESTful API)
Storage Format	JSON
Visualization	Dynamic Graph Rendering via Vis.js
Algorithmic Concepts	BFS, DFS (Uninformed Search)
Tools	Visual Studio Code, Chrome, Python 3.x

10. Functional Requirements

ID	Requirement	Description
FR-01	Add City	Add a new node representing a city.
FR-02	Add Connection	Add a road with a specific distance.
FR-03	Select Algorithm	Choose BFS or DFS for exploration.
FR-04	Compute Path	Find and visualize the route between start and end nodes.
FR-05	Display Result	Show explored nodes, path cost, and traversal order.
FR-06	Save Graph	Save graph as a JSON file.
FR-07	Load Graph	Load graph from saved files.
FR-08	Clear Graph	Remove all nodes and edges.
FR-09	Toggle Physics	Control animation and layout movement.
FR-10	Error Handling	Handle missing nodes or invalid connections gracefully.

11. Non-Functional Requirements

Category	Description
Performance	Should process and visualize paths for up to 100 nodes in under 1 second.
Usability	Intuitive UI, color-coded feedback, minimal learning curve.
Scalability	Modular backend for adding new algorithms.
Reliability	Prevents data corruption and manages file access safely.
Portability	Runs in any modern web browser with JavaScript enabled.
Maintainability	Code separated into backend (Flask) and frontend (Vis.js) modules for easier updates.

12. Real-World Relevance

This project represents how **AI algorithms are applied in real-world city routing systems**, such as:

- **Google Maps** route exploration (shortest path search).
- **Autonomous Vehicle Navigation** (AI decision-making in city routes).
- **Smart Traffic Management Systems** (dynamic graph traversal).
- **Logistics and Delivery Optimization** (finding optimal delivery routes).

By connecting BFS and DFS with real city maps, this project demonstrates how **AI powers decision-making systems** that mimic human reasoning in urban environments.

13. Future Enhancements

1. **Heuristic Search Algorithms**
 - Implement **Dijkstra's**, **A***, and **Greedy Search**.
2. **Cost Optimization**
 - Find least-cost routes instead of shortest step routes.
3. **Dynamic Environment**
 - Simulate road closures or varying travel costs.
4. **Step-by-Step Control**
 - Add pause, play, and next-step controls for algorithm animations.
5. **Database Integration**
 - Replace JSON files with SQLite or MongoDB.
6. **3D Visualization**
 - Integrate Mapbox or Leaflet for geographical accuracy.
7. **Mobile Version**
 - Responsive design for smartphones and tablets.

14. Conclusion

The **AI-Based City Pathfinding and Visualization System** demonstrates how Artificial Intelligence algorithms can solve real-world problems like city navigation. By combining **graph theory**, **BFS**, and **DFS**, the project shows how AI explores possible routes, determines connectivity, and finds paths between cities in a human-like manner.

Through visual animation, users can observe how these algorithms make decisions step by step, turning abstract AI concepts into an understandable and interactive experience. This bridges the gap between theory and practice, making it easier to learn how search strategies work in real environments.

The project also reflects how similar algorithms are used in real applications such as **GPS route planning**, **autonomous vehicle navigation**, and **smart city systems**. In the future, the system can be enhanced with **heuristic algorithms** like **Dijkstra's** and **A*** to handle more complex and cost-efficient pathfinding.

Overall, the project fulfills its goal of visualizing AI pathfinding while demonstrating the real-world significance of search algorithms in intelligent transportation and decision-making systems.